0567. Permutation in String

Medium · Sliding Window

leetcode.com/problems/permutation-in-string

Problem

- Return true if some permutation of s1 is a substring of s2.
- Example: s1 = "ab", s2 = "eidbaooo" → true ("ba").

Intuition

- A permutation of s1 is any window of length |s1| with identical letter counts.
- Slide a fixed-size window over s2 and keep a count of how many of the 26 letters currently match.

Brute Force

- For every window, compare 26 counts: O(26 n). Fine too, but the "matches" counter makes each step O(1).

Optimal Approach

- Need[26] from s1, have[26] for the window.
- Matches = number of letters where need == have.
- When a char enters/leaves, update have and adjust matches only for that letter.
- If matches == 26, return true.

Worked Example

```
s1 = "ab", s2 = "eidbaooo".

window "ei" -> no
window "id" -> no
window "db" -> no
window "ba" -> counts a1 b1 match need -> true
```

Java Solution

```java
public class PermutationInString {
    public boolean checkInclusion(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        if (m > n) return false;
        int[] diff = new int[26]; // need - have
        for (int i = 0; i < m; i++) {
            diff[s1.charAt(i) - 'a']++;
            diff[s2.charAt(i) - 'a']--;
        }
        int zeros = 0;
        for (int d : diff) if (d == 0) zeros++;
        if (zeros == 26) return true;
        for (int r = m; r < n; r++) {
            zeros = update(diff, s2.charAt(r) - 'a', -1, zeros);
            zeros = update(diff, s2.charAt(r - m) - 'a', +1, zeros);
            if (zeros == 26) return true;
        }
        return false;
    }

    private int update(int[] diff, int c, int delta, int zeros) {
        if (diff[c] == 0) zeros--;
        diff[c] += delta;
        if (diff[c] == 0) zeros++;
        return zeros;
    }
}
```

Complexity

- Time: O(|s1| + |s2|). Space: O(26).

Pitfalls

- If |s1| > |s2| return false immediately.